

Haute Ecole Informatique (HEI)

SÉCU 2 EXAMEN 1 Partie pratique

DEADLINE: **17 JANVIER 2025 23:59**

Tout document autorisé. Tout internet autorisé, y compris ChatGpt et autre GenAI.

"Vulnerabilities in code often stem from edge cases, where input or state deviate from the developer's expectations."

**Analyse de sécurité et détection de vulnérabilités
dans l'implémentation SFTP de WOLSSH**

Contexte :

L'objectif de cet examen est d'analyser et d'identifier les vulnérabilités dans le code SFTP de WOLSSH, un serveur SFTP basé sur **wolfSSH**, et de **provoquer des erreurs ASAN** pour détecter ces failles à travers un script Python utilisant la bibliothèque **Paramiko** pour se connecter à un serveur SFTP vulnérable.

L'examen se décompose en deux parties :

1. **Identification des failles** : Analyse du code pour détecter les vulnérabilités potentielles dans les fonctions `wolfSSH_SFTP_RecvWrite`, `wolfSSH_SFTP_RecvRead` et `wolfSSH_SFTP_RecvRealPath` de WOLSSH.
2. **Détection des failles via ASAN** : Écriture d'un script Python qui se connecte à un serveur SFTP WOLSSH vulnérable et provoque des erreurs de mémoire détectées par **AddressSanitizer (ASAN)**, ce qui permet de démontrer la présence des vulnérabilités sans exploitation complète.

Partie 1 : Identification des failles

Les fonctions suivantes de WOLSSH doivent être examinées pour y détecter des failles de sécurité :

1. `wolfSSH_SFTP_RecvWrite` :

Cette fonction gère les commandes **write** dans le protocole SFTP. Elle doit être analysée pour vérifier les vulnérabilités liées à la validation insuffisante des entrées ou des manipulations incorrectes de la mémoire.

```
int wolfSSH_SFTP_RecvWrite(WOLFSSH* ssh, int reqId, byte* data, word32
maxSz)
#ifdef USE_WINDOWS_API
{
    WFD    fd;
    word32 sz;
    int    ret  = WS_SUCCESS;
    word32 idx  = 0;
    word32 ofst[2] = {0,0};

    word32 outSz = 0;
    byte*  out   = NULL;

    char  suc[] = "Write File Success";
    char  err[] = "Write File Error";
    char* res   = suc;
    byte  type  = WOLFSSH_FTP_OK;

    if (ssh == NULL) {
```

```

return WS_BAD_ARGUMENT;
}

WLOG(WS_LOG_SFTP, "Receiving WOLFSSH_FTP_WRITE");

/* get file handle */
ato32(data + idx, &sz); idx += UINT32_SZ;
if (sz + idx > maxSz || sz > WOLFSSH_MAX_HANDLE) {
WLOG(WS_LOG_SFTP, "Error with file handle size");
res = err;
type = WOLFSSH_FTP_FAILURE;
ret = WS_BAD_FILE_E;
}

if (ret == WS_SUCCESS) {
WMMEMSET((byte*)&fd, 0, sizeof(WFD));
WMMEMCPY((byte*)&fd, data + idx, sz); idx += sz;

/* get offset into file */
ato32(data + idx, &ofst[1]); idx += UINT32_SZ;
ato32(data + idx, &ofst[0]); idx += UINT32_SZ;

/* get length to be written */
ato32(data + idx, &sz); idx += UINT32_SZ;

ret = WPWRITE(fd, data + idx, sz, ofst);
if (ret < 0) {
#ifdef WOLFSSL_NUCLEUS && defined(DEBUG_WOLFSSH)
    if (ret == NUF_NOSPC) {
        WLOG(WS_LOG_SFTP, "Ran out of memory");
    }
#endif
WLOG(WS_LOG_SFTP, "Error writing to file");
res = err;
type = WOLFSSH_FTP_FAILURE;
ret = WS_INVALID_STATE_E;
}
else {
    ret = WS_SUCCESS;
}
}

if (wolfSSH_SFTP_CreateStatus(ssh, type, reqId, res, "English",
NULL,
    &outSz) != WS_SIZE_ONLY) {
return WS_FATAL_ERROR;
}

```

```

    }
    out = (byte*)WMALLOC(outSz, ssh->ctx->heap, DYNTYPE_BUFFER);
    if (out == NULL) {
        return WS_MEMORY_E;
    }
    if (wolfSSH_SFTP_CreateStatus(ssh, type, reqId, res, "English",
out,
                                &outSz) != WS_SUCCESS) {
        WFREE(out, ssh->ctx->heap, DYNTYPE_BUFFER);
        return WS_FATAL_ERROR;
    }

    /* set send out buffer, "out" is taken by ssh */
    wolfSSH_SFTP_RecvSetSend(ssh, out, outSz);
    return ret;
}

```

2. wolfSSH_SFTP_RecvRead :

Cette fonction gère les commandes **read** dans le protocole SFTP.

```

int wolfSSH_SFTP_RecvRead(WOLFSSH* ssh, int reqId, byte* data, word32
maxSz)
#ifdef USE_WINDOWS_API
{
    WFD    fd;
    word32 sz;
    int    ret;
    word32 idx = 0;
    word32 ofst[2] = {0, 0};

    byte*  out;
    word32 outSz;

    char*  res  = NULL;
    char  err[] = "Read File Error";
    char  eof[] = "Read EOF";
    byte  type = WOLFSSH_FTP_FAILURE;

    if (ssh == NULL) {
        return WS_BAD_ARGUMENT;
    }

    WLOG(WS_LOG_SFTP, "Receiving WOLFSSH_FTP_READ");

    /* get file handle */

```

```

    ato32(data + idx, &sz); idx += UINT32_SZ;
    if (sz + idx > maxSz || sz > WOLFSSH_MAX_HANDLE) {
        return WS_BUFFER_E;
    }
    WMEMSET((byte*)&fd, 0, sizeof(WFD));
    WMEMCPY((byte*)&fd, data + idx, sz); idx += sz;

    /* get offset into file */
    ato32(data + idx, &ofst[1]); idx += UINT32_SZ;
    ato32(data + idx, &ofst[0]); idx += UINT32_SZ;

    /* get length to be read */
    ato32(data + idx, &sz);

    /* read from handle and send data back to client */
    out = (byte*)WMALLOC(sz + WOLFSSH_SFTP_HEADER + UINT32_SZ,
        ssh->ctx->heap, DYNTYPE_BUFFER);
    if (out == NULL) {
        return WS_MEMORY_E;
    }

    ret = WPREAD(fd, out + UINT32_SZ + WOLFSSH_SFTP_HEADER, sz, ofst);
    if (ret < 0 || (word32)ret > sz) {
        WLOG(WS_LOG_SFTP, "Error reading from file");
        res = err;
        type = WOLFSSH_FTP_FAILURE;
        ret = WS_BAD_FILE_E;
    }
    else {
        outSz = (word32)ret + WOLFSSH_SFTP_HEADER + UINT32_SZ;
    }

    /* eof */
    if (ret == 0) {
        WLOG(WS_LOG_SFTP, "Error reading from file, EOF");
        res = eof;
        type = WOLFSSH_FTP_EOF;
        ret = WS_SUCCESS; /* end of file is not fatal error */
    }

    if (res != NULL) {
        if (wolfSSH_SFTP_CreateStatus(ssh, type, reqId, res, "English",
            NULL,
                &outSz) != WS_SIZE_ONLY) {
            WFREE(out, ssh->ctx->heap, DYNTYPE_BUFFER);
            return WS_FATAL_ERROR;
        }
    }

```

```

    }
    if (outSz > sz) {
        /* need to increase buffer size for holding status packet */
        WFREE(out, ssh->ctx->heap, DYNTYPE_BUFFER);
        out = (byte*)WMALLOC(outSz, ssh->ctx->heap, DYNTYPE_BUFFER);
        if (out == NULL) {
            return WS_MEMORY_E;
        }
    }
    if (wolfSSH_SFTP_CreateStatus(ssh, type, reqId, res, "English",
out,
                                &outSz) != WS_SUCCESS) {
        WFREE(out, ssh->ctx->heap, DYNTYPE_BUFFER);
        return WS_FATAL_ERROR;
    }
}
else {
    SFTP_CreatePacket(ssh, WOLFSSH_FTP_DATA, out, outSz, NULL, 0);
}

/* set send out buffer, "out" is taken by ssh */
wolfSSH_SFTP_RecvSetSend(ssh, out, outSz);
return ret;
}

```

3. wolfSSH_SFTP_RecvRealPath :

Cette fonction traite les commandes **realpath**, permettant de récupérer le chemin réel d'un fichier ou d'un répertoire.

```

static int wolfSSH_SFTP_RecvRealPath(WOLFSSH* ssh, int reqId, byte*
data,
    int maxSz)
{
    WS_SFTP_FILEATRB atr;
    char r[WOLFSSH_MAX_FILENAME];
    word32 rSz;
    word32 lidx = 0;
    word32 i;
    int ret;
    byte* out;
    word32 outSz = 0;

    WLOG(WS_LOG_SFTP, "Receiving WOLFSSH_FTP_REALPATH");
}

```

```

    if (ssh == NULL) {
        WLOG(WS_LOG_SFTP, "Bad argument passed in");
        return WS_BAD_ARGUMENT;
    }

    if (maxSz < UINT32_SZ) {
        /* not enough for an ato32 call */
        return WS_BUFFER_E;
    }

    ato32(data + lidx, &rSz);
    if (rSz > WOLFSSH_MAX_FILENAME || (int)(rSz + UINT32_SZ) > maxSz)
{
    return WS_BUFFER_E;
}
    lidx += UINT32_SZ;
    WMEMCPY(r, data + lidx, rSz);
    r[rSz] = '\0';

    /* get working directory in the case of receiving non absolute
path */
    if (r[0] != '/' && r[1] != ':') {
        char wd[WOLFSSH_MAX_FILENAME];

        WMEMSET(wd, 0, WOLFSSH_MAX_FILENAME);
        if (ssh->sftpDefaultPath) {
            XSTRNCPY(wd, ssh->sftpDefaultPath, WOLFSSH_MAX_FILENAME -
1);
        }
        else {
#ifdef USE_WINDOWS_API
            if (WGETCWD(ssh->fs, wd, WOLFSSH_MAX_FILENAME) == NULL) {
                WLOG(WS_LOG_SFTP, "Unable to get current working
directory");
            }
            if (wolfSSH_SFTP_CreateStatus(ssh,
WOLFSSH_FTP_FAILURE, reqId,
                "Directory error", "English", NULL, &outSz)
                != WS_SIZE_ONLY) {
                return WS_FATAL_ERROR;
            }
            out = (byte*) WMALLOC(outSz, ssh->ctx->heap,
DYNTYPE_BUFFER);
            if (out == NULL) {
                return WS_MEMORY_E;
            }

```

```

        if (wolfSSH_SFTP_CreateStatus(ssh,
WOLFSSH_FTP_FAILURE, reqId,
        "Directory error", "English", out, &outSz)
        != WS_SUCCESS) {
            WFREE(out, ssh->ctx->heap, DYNTYPE_BUFFER);
            return WS_FATAL_ERROR;
        }
        /* take over control of buffer */
        wolfSSH_SFTP_RecvSetSend(ssh, out, outSz);
        return WS_BAD_FILE_E;
    }
#endif
}
WSTRNCAT(wd, "/", WOLFSSH_MAX_FILENAME);
WSTRNCAT(wd, r, WOLFSSH_MAX_FILENAME);
WMEMCPY(r, wd, WOLFSSH_MAX_FILENAME);
}

if ((ret = wolfSSH_CleanPath(ssh, r)) < 0) {
    return WS_FATAL_ERROR;
}
rSz = (word32)ret;

/* For real path remove ending case of /.
 * Lots of peers send a '.' wanting a return of the current
absolute path
 * not the absolute path + .
 */
if (r[rSz - 2] == WS_DELIM && r[rSz - 1] == '.') {
    r[rSz - 1] = '\0';
    rSz -= 1;
}

/* for real path always send '/' chars */
for (i = 0; i < rSz; i++) {
    if (r[i] == WS_DELIM) r[i] = '/';
}
WLOG(WS_LOG_SFTP, "Real Path Directory = %s", r);

/* send response */
outSz = WOLFSSH_SFTP_HEADER + (UINT32_SZ * 3) + (rSz * 2);
WMEMSET(&atr, 0, sizeof(WS_SFTP_FILEATR));
outSz += SFTP_AttributesSz(ssh, &atr);
lidx = 0;

/* reuse state buffer if large enough */

```



```

    out = (outSz > (word32)maxSz)?
        (byte*)WMALLOC(outSz, ssh->ctx->heap, DYNTYPE_BUFFER) :
        wolfSSH_SFTP_RecvGetData(ssh);
    if (out == NULL) {
        return WS_MEMORY_E;
    }

    SFTP_SetHeader(ssh, reqId, WOLFSSH_FTP_NAME,
        outSz - WOLFSSH_SFTP_HEADER, out);
    lidx += WOLFSSH_SFTP_HEADER;

    /* set number of files */
    c32toa(1, out + lidx); lidx += UINT32_SZ; /* only sending one file
name */

    /* set file name size and string */
    c32toa(rSz, out + lidx); lidx += UINT32_SZ;
    WMEMCPY(out + lidx, r, rSz); lidx += rSz;

    /* set long name size and string */
    c32toa(rSz, out + lidx); lidx += UINT32_SZ;
    WMEMCPY(out + lidx, r, rSz); lidx += rSz;

    /* set attributes */
    SFTP_SetAttributes(ssh, out + lidx, outSz - lidx, &atr);

    /* set send out buffer, "out" buffer is taken over by "ssh" */
    wolfSSH_SFTP_RecvSetSend(ssh, out, outSz);
    return WS_SUCCESS;
}

```

Partie 2 : Détection des failles via ASAN

Une fois les failles identifiées dans le code de **WOLSSH**, l'objectif de cette partie est de **provoquer des erreurs ASAN** pour confirmer la présence de ces vulnérabilités, en utilisant un script Python.

Objectif :

Vous devrez créer un script Python utilisant la bibliothèque **Paramiko** pour vous connecter à un serveur SFTP vulnérable (WOLSSH), et provoquer des erreurs mémoire qui seront captées par **AddressSanitizer** (ASAN). <https://clang.llvm.org/docs/AddressSanitizer.html>

AddressSanitizer (ASAN) est un outil de détection de bugs en mémoire, principalement utilisé pour identifier des problèmes tels que les dépassements de tampon (buffer overflows), les dépassements de pile, les erreurs d'accès à des zones de mémoire déjà libérées (use-after-free), et les fuites de mémoire.

Fonctionnement d'ASAN :

1. Instrumentation du code :

- ASAN fonctionne en instrumentant le code source pendant la compilation. Cela signifie qu'il insère des vérifications supplémentaires autour des opérations de mémoire pour détecter des erreurs à l'exécution.
- Les zones de mémoire sont entourées de "gardes" (canaries) qui permettent de détecter des écritures hors des limites prévues.

2. Détection des erreurs à l'exécution :

- Lorsque le programme instrumenté est exécuté, ASAN surveille les accès à la mémoire et intercepte les accès non valides, tels que l'accès à des tampons trop petits, l'utilisation de pointeurs invalides, ou l'accès à de la mémoire déjà libérée.
- En cas de détection d'une violation, ASAN génère un rapport détaillé, incluant l'emplacement précis de l'erreur, la pile d'appels, et le type de violation.

Types de bugs détectés par ASAN :

1. Buffer Overflow :

- Écriture ou lecture en dehors des limites allouées d'un tableau ou d'un tampon.

2. Heap Overflow :

- Accès à des zones de mémoire allouées dynamiquement au-delà de leur taille définie.

3. Use-After-Free :

- Utilisation de la mémoire après qu'elle a été libérée.

4. Double-Free :

- Libération de la même zone mémoire plus d'une fois.

5. Stack Overflow :

- Écriture en dehors des limites d'une variable locale.

6. Global Overflow :

- Écriture ou lecture en dehors des limites d'une variable globale ou statique.

Exemple de rapport ASAN :

Lorsqu'une erreur est détectée, ASAN produit un message qui ressemble à ceci :

```
==12345==ERROR: AddressSanitizer: heap-buffer-overflow on address
0x602000000050 at pc 0x55555555517d bp 0x7fffffffe430 sp 0x7fffffffe420
WRITE of size 4 at 0x602000000050 thread T0
```

```
#0 0x5555555517c in function_name (/path/to/program+0x117c)
#1 0x5555555528f in main (/path/to/program+0x128f)
#2 0x7ffff7a04b96 in __libc_start_main
(/lib/x86_64-linux-gnu/libc.so.6+0x21b96)
#3 0x555555550a9 in _start (/path/to/program+0x10a9)
```

Ce rapport donne des informations détaillées sur :

- Le type d'erreur (ici, un **heap-buffer-overflow**).
- L'adresse mémoire impliquée.
- La taille de l'accès problématique.
- La pile d'appels qui a conduit à l'erreur.

Avantages d'ASAN :

- **Efficacité** : ASAN détecte rapidement les erreurs de mémoire difficiles à trouver par les tests manuels.
- **Facilité d'utilisation** : Il s'intègre facilement aux compilateurs modernes comme **GCC** et **Clang**.
- **Rapports détaillés** : ASAN fournit des messages d'erreur clairs qui aident les développeurs à localiser et corriger les bugs.

Inconvénients d'ASAN :

- **Surcharge de performance** : L'instrumentation du code et la surveillance constante des accès mémoire entraînent une réduction de la vitesse d'exécution et une augmentation de l'utilisation de la mémoire.
- **Faux positifs** : Dans certains cas, ASAN peut signaler des erreurs qui ne sont pas des bugs critiques, nécessitant un tri des rapports.

En résumé, **ASAN** est un outil puissant pour renforcer la sécurité des applications en détectant des bugs de mémoire courants qui pourraient être exploités pour compromettre la stabilité ou la sécurité d'un programme.

Pour compiler **wolfSSL** et **wolfSSH** avec le support d'ASAN (AddressSanitizer), les options de débogage **-g** et **-O0**, et les versions spécifiées (**wolfSSL** stable et **wolfSSH 1.4.6**), suivez ces étapes détaillées :

- Télécharger l'image VDI ubuntu server 24.04

<https://sourceforge.net/projects/osboxes/files/v/vb/59-U-u-svr/24.04/64bit.7z/download>

- Importer l'image VDI sous virtualbox
 - **Username:** osboxes

- **Password:** osboxes.org
-
- Installer et configurer un serveur OpenSSH sous Ubuntu
- configurer **VirtualBox** afin de permettre à l'hôte de se connecter à une machine virtuelle (guest) via SSH

```
ssh -p 2222 osboxes@localhost
```

Étape 1 : Préparation de l'Environnement

Assurez-vous que votre système dispose des outils nécessaires pour la compilation :

```
sudo apt-get update
sudo apt-get install -y build-essential cmake git autoconf libtool
pkg-config gdb
```

Étape 2 : Cloner les Dépôts

Cloner les versions stables des dépôts **wolfSSL** et **wolfSSH** :

```
# get wolfSSL
wget
https://github.com/wolfSSL/wolfssl/archive/refs/tags/v5.7.6-stable.tar.g
z

# get wolfSSH
Wget
https://github.com/wolfSSL/wolfssh/archive/refs/tags/v1.4.6-stable.tar.g
z
```

Étape 3 : Compilation de wolfSSL

Compilez wolfSSL

```
tar -xf v5.7.6-stable.tar.gz
cd wolfssl-5.7.6-stable/
```

```
./autogen.sh
./configure --enable-ssh
make
```

```
sudo make install
sudo ldconfig
```

Étape 4 : Compilation de wolfSSH avec ASAN

Compilez **wolfSSH** en utilisant la version de **wolfSSL** compilée avec ASAN :

```
tar -xf v1.4.6-stable.tar.gz
cd wolfssh-1.4.6-stable/
./autogen.sh
CFLAGS="-fsanitize=address -g -O0" ./configure --enable-sftp
make
```

Étape 5 : Lancement du Serveur SFTP et du Client

Pour tester le serveur **wolfSSH** avec SFTP et vous connecter à l'aide d'un client, suivez ces étapes.

1. Lancer le Serveur SFTP avec wolfSSH

Assurez-vous que **wolfSSH** est compilé et installé correctement. Ensuite, vous pouvez lancer le serveur SFTP en utilisant l'exemple fourni dans les sources de **wolfSSH**.

```
examples/echoserver/echoserver -f
```

Par défaut, le serveur écoutera sur le port 22222

2. Lancer le Client SFTP avec Paramiko

Une fois le serveur SFTP lancé, vous pouvez utiliser un script Python avec la bibliothèque **Paramiko** pour vous connecter en tant que client.

```
sudo apt install python3-pip
sudo apt install python3.12-venv
python3 -m venv sftp_env
source sftp_env/bin/activate
```

```
sudo apt install nano
pip install paramiko
```

```
import paramiko
import logging

# Activer les logs de débogage
#logging.basicConfig(level=logging.DEBUG)

# Détails de connexion
hostname = "127.0.0.1"
port = 22222
username = "jill"
password = "upthehill"

# Créer un client SFTP
transport = paramiko.Transport((hostname, port))
transport.connect(username=username, password=password)

sftp = paramiko.SFTPClient.from_transport(transport)

# Lister les fichiers dans le répertoire racine
files = sftp.listdir('.')
print("Fichiers sur le serveur SFTP:", files)

# Fermer la connexion
sftp.close()
transport.close()
```

```
python sftp_client.py
```

```
Fichiers sur le serveur SFTP: ['sftp_env', 'wolfssh-config', 'libtool',
'examples', 'src', 'config.log', 'README.md', 'gencertbuf.pl',
'Makefile.am',
'm4', 'README', 'aminclude.am', 'wolfssh', 'configure.ac', 'configure',
'notes.md', 'Makefile', 'config.status', '.gitignore', 'autom4te.cache',
```

```
'scri  
pts', 'ChangeLog.md', 'autogen.sh', 'build-aux', 'tests', 'keys', 'ide',  
'sftp_client.py', 'Makefile.in', 'aclocal.m4', 'LICENSING']
```

Pour déboguer un processus en cours d'exécution avec **gdb**, vous pouvez utiliser l'option **-p** pour attacher **gdb** au processus cible. Voici les étapes détaillées :

Étape 1 : Trouver l'ID du Processus (PID)

Avant d'attacher **gdb**, vous devez connaître l'identifiant du processus que vous souhaitez déboguer.

- Utilisez la commande **ps** pour trouver le PID du processus :

```
ps aux | grep nom_du_processus
```

Par exemple, si vous voulez attacher à un serveur SFTP en cours d'exécution :

```
ps aux | grep echoserver
```

Étape 2 : Attacher gdb au Processus

Une fois que vous avez le PID, vous pouvez attacher **gdb** au processus avec la commande suivante :

```
gdb -p pid
```

```
gdb -p 1234
```

Livrables

1. Rapport d'Analyse et de Détection des Vulnérabilités

- **Description des Vulnérabilités** : Liste des fonctions vulnérables (e.g., wolfSSH_SFTP_RecvWrite, wolfSSH_SFTP_RecvRead, wolfSSH_SFTP_RecvRealPath)

2. Script Python de Test + Capture Video

Script Python avec Paramiko : Fournissez un script Python utilisant **Paramiko** pour se connecter au serveur SFTP vulnérable et exécuter des actions qui déclenchent les erreurs ASAN.

3. Sorties ASAN + Capture Video

- **Logs ASAN** : Incluez les sorties des logs d'ASAN montrant les erreurs détectées, comme les dépassements de tampon ou les erreurs de gestion de la mémoire.

4. Instructions de Reproduction

- **Guide d'Installation et d'Exécution** : Fournissez un guide pas-à-pas pour installer et configurer l'environnement (docker, virtualenv, etc.) et exécuter les tests. Assurez-vous que les instructions permettent de reproduire les erreurs ASAN.
- **Dockerfile** : Si vous utilisez Docker, incluez le fichier Dockerfile pour construire l'image de l'environnement de test.

5. Conclusion et Recommandations

- **Résumé des Résultats** : Résumez les failles découvertes et leur gravité.
- **Recommandations** : Suggérez des correctifs pour les vulnérabilités détectées, comme l'amélioration des validations d'entrée ou le renforcement de la gestion de la mémoire.

À : michael@mail.hei.school

Objet : SECU2_EXAM_1_Reference_STD

Fichier .zip contenant tous les livrables.

DEADLINE: 17 JANVIER 2025 23:59

Tout document autorisé. Tout internet autorisé, y compris ChatGpt et autre GenAI.

"Vulnerabilities in code often stem from edge cases, where input or state deviate from the developer's expectations."